ARTIFICIAL INTELLIGENCE MINI PROJECT (CS2203)

-SUBMITTED BY
2301CS31_2301CS74_2301CS75_2301CS88_2302CS05

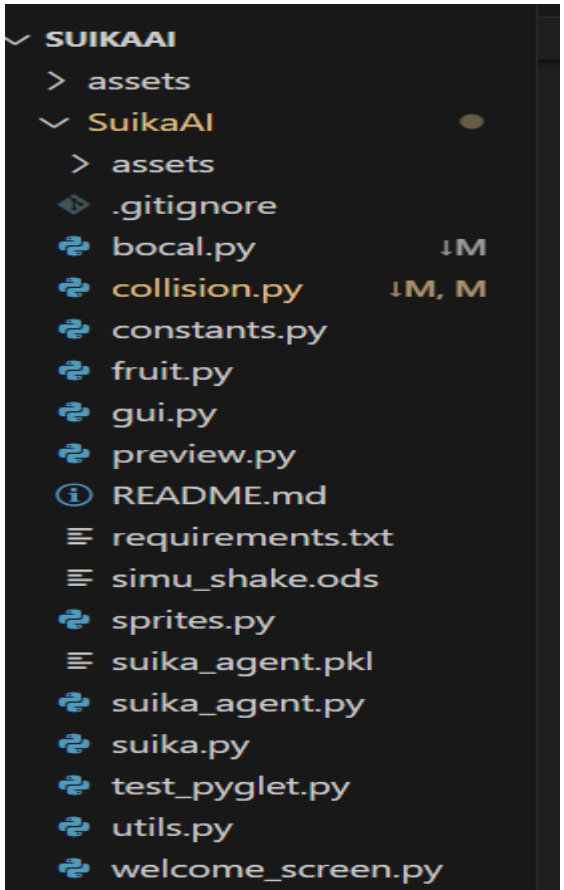
WELCOME TO

LET’s START

ABOUT THE GAME

Suika Game, also known as the "Watermelon Game," is a Japanese puzzle video game developed by Aladdin X. It combines elements of falling and merging puzzle games, where players drop fruits into a box, and when two identical fruits touch, they merge into a larger fruit. The objective is to create the largest possible fruit while preventing the box from overflowing.

NOW LETS UNDERSTAND SOME MAIN FILES OF OUR PROJECT AND HOW THE CODES ARE EXACTLY WORKING



THIS IS A CURATED LIST OF ALL FILES IN OUR PROJECT

BOCAL.PY

LIKELY HANDLES THE PHYSICS OF BOUNCING FRUITS, SUCH AS APPLYING GRAVITY, FRICTION, AND COLLISION RESPONSES.
IT MAY DEFINE FUNCTIONS TO SIMULATE HOW FRUITS BEHAVE WHEN THEY HIT EACH OTHER OR THE CONTAINER.

THIS CODE SIMULATES A **GAME CONTAINER**:

- WALLS DEFINE **BOUNDARIES**.
- OBJECTS ARE **DROPPED** INTO THE CONTAINER.
- **SHAKING & TUMBLING** ALLOW INTERACTIVE PHYSICS-BASED MOVEMENT.
- USES **PYMUNK** FOR REALISTIC PHYSICS SIMULATION.

```
class Bocal(object):
    """ Utility to create the walls of the game space (space).
    """
    def __init__(self, space, center, bocal_w, bocal_h):
        # Create a static body for the container
        self._body = pm.Body(body_type=pm.Body.STATIC)  # Changed to STATIC
        self._position_ref = center
        self._width_ref = bocal_w
        self._height_ref = bocal_h
        self._body.position = center  # Set initial position immediately
        space.add(self._body)

        self._walls = _make_walls(space, width=bocal_w, height=bocal_h)
        self._space = space
        self._maxline = self._walls[MAXLINE]
        self._dropzone = DropZone(bocal_body=self._body, width=bocal_w, height=bocal_h)
        self.reset()
```

COLLISION.PY

DEALS WITH DETECTING WHEN TWO FRUITS TOUCH. THIS IS CRUCIAL FOR MERGING IDENTICAL FRUITS IN SUIKA GAME. THE FILE CONTAINS FUNCTIONS FOR CHECKING OVERLAP OR PROXIMITY BETWEEN GAME OBJECTS.

INSIDE COLLSION HELPER CLASS WE HAVE

COLLISION DETECTION FUNCTIONS
COLLISION PROCESSING FUNCTIONS
COLLISION HANDLERS SETUP

1. **Detects** collisions between objects.
2. **Handles fruit merging**
3. **Triggers visual effects.**
4. **Schedules new fruit spawning.**
5. **Implements event-based physics handling using Pymunk.**

```
class CollisionHelper(object):
    """ Contains the callback called by pymunk for each collision
    and the algorithms for selecting fruits to merge and create
"""
    def __init__(self, space):
        self.reset()
        self.setup_handlers( space )


    def reset( self ):
        self._collisions_fruits = []
        self._actions = []

    def collision_fruit( self, arbiter ):
        """  Callback for pymunk collision_handler
        """
        s0 = arbiter.shapes[0]
        s1 = arbiter.shapes[1]

        shapes = arbiter.shapes
        assert( len(shapes)==2 ), " WTF ???"
        assert( s0.fruit.kind == s1.fruit.kind )
        self._collisions_fruits.append( (s0.fruit, s1.fruit) )
        return True


    def collision_first_drop( self, arbiter ):
        """ Called when a fruit falls on another for the first time after b
```

CONSTANTS.PY

STORES FIXED VALUES USED THROUGHOUT THE GAME, SUCH AS FRUIT SIZES,GRAVITY STRENGTH, SCREEN DIMENSIONS, COLORS, AND OTHER PARAMETERS. THIS ENSURES MAINTAINABILITY BY KEEPING KEY VALUES IN ONE PLACE.

This file defines all constants used in your game:

1.Window & UI settings → Window size, margins.

2.Physics settings → Gravity, elasticity, and movement.

3.Game timing & animations → Merging, explosion, and blinking effects.

4.Collision types & filtering → Detects fruit merging, wall collisions, game over conditions.

This file acts like a game configuration sheet. If you want to adjust gameplay, modify values here.

```
############ Application Appearance #############
WINDOW_HEIGHT = 1800
WINDOW_WIDTH  = 1400
BOCAL_MARGIN_TOP = 200
BOCAL_MARGIN_BOTTOM = 50
BOCAL_MARGIN_SIDE = 150
GUI_FONT_SIZE = 25
GUI_TOP_MARGIN = 10
NEXT_FRUIT_Y_POS = BOCAL_MARGIN_TOP//2   # pixels from top
PREVIEW_Y_POS =  90    # pixels from top
PREVIEW_SPRITE_SIZE = 50
PREVIEW_SLOT_SIZE = 70
PREVIEW_COUNT = 3

################ Appearance of container ############
WALL_THICKNESS = 20
WALL_COLOR = (205,200,255,255)
REDLINE_TOP_MARGIN = 170
REDLINE_THICKNESS = 2
REDLINE_COLOR= (255,20,20,255)
BOCAL_MIN_WIDTH=300
BOCAL_MIN_HEIGHT=400

############ Physical Simulation #############
PYMUNK_INTERVAL = 1 / 120.0
FRICTION = 1.0
GRAVITY = -981
INITIAL_VELOCITY = 1200
```

FRUIT.PY

DEFINES THE FRUIT OBJECTS, THEIR ATTRIBUTES (SIZE, TYPE, IMAGES),AND BEHAVIORS (HOW THEY MOVE, HOW THEY MERGE, ETC.).IT MAY ALSO INCLUDE METHODS FOR DRAWING AND UPDATING FRUIT PROPERTIES.

THIS CODE CONTAINS

1.FRUIT DEFINITIONS
2.GAME MODES
3.COLLISION SETTINGS
4.FRUIT OBJECTCREATION
5.DROPPING AND MERGING OF FRUITS

```
_FRUITS_DEF_ORIGINAL = [
    # The rank in this list serves as kind/points/collision_type;
    # Value 0 is reserved for types without handlers.
    None,
    {'mass':5,  'radius':30,  'name':'cerise' },
    {'mass':7,  'radius':40,  'name':'fraise' },
    {'mass':10, 'radius':55,  'name':'prune' },
    {'mass':12, 'radius':70,  'name':'abricot' },
    {'mass':15, 'radius':90,  'name':'orange' },
    {'mass':20, 'radius':120, 'name':'tomate' },
    {'mass':25, 'radius':130, 'name':'pamplemousse' },
    {'mass':30, 'radius':150, 'name':'pomme' },
    {'mass':37, 'radius':190, 'name':'ananas' },
    {'mass':40, 'radius':230, 'name':'melon' },
    {'mass':50, 'radius':200, 'name':'pasteque' },
]
```

GUI.PY

MANAGES THE GRAPHICAL USER INTERFACE (GUI) OF THE GAME, INCLUDING MENUS, BUTTONS, AND ANY USER INTERACTIONS OUTSIDE THE MAIN GAMEPLAY. IT MIGHT ALSO HANDLE DISPLAYING SCORES AND MESSAGES.

✅ Uses Pymunk for physics (falling, bouncing, merging).
✅ Uses Pyglet for rendering (sprites, animations).
✅ Implements fruit merging mechanics (combine small fruits into bigger ones).
✅ Tracks game states (dropping, merging, waiting).
✅ Handles collisions, gravity, and off-screen removal.

```
class GUI(object):
    def __init__( self, window_width, window_height) :

        # textes en haut
        self._label_topleft = TopLeftLabel(window_width, window_height)
        self._label_center = CenterLabel(window_width, window_height)
        self._label_topright = TopRightLabel(window_width, window_height)
        self._gameover = GameOverSprite( window_width, window_height )
        self._gameover_mask = GameOverMask( window_width, window_height)
        self._resizables = [self._gameover,
                            self._gameover_mask,
                            self._label_topleft,
                            self._label_center,
                            self._label_topright ]

    def on_resize(self, width, height):
        for item in self._resizables:
            item.on_resize(width, height)

    def update_label(self, label, text):
        if(label==TOP_LEFT): self._label_topleft.text = text
        if(label==TOP_CENTER): self._label_center.text = text
        if(label==TOP_RIGHT): self._label_topright.text = text

    def update_dict( self, texts ):
```

PREVIEW.PY

HANDLES PREVIEWING THE NEXT FRUIT TO BE DROPPED, SIMILAR TO HOW TETRIS SHOWS UPCOMING PIECES.IT MIGHT BE RESPONSIBLE FOR RENDERING A SMALL PREVIEW BOX IN THE UI.

1.Maintains a preview queue of upcoming fruits.
2.Ensures smooth shifting animations.
3.Updates the position of previewed fruits dynamically.
4.Adjusts the queue when a fruit is used.
This is essential for the Suika game's UI, allowing players to anticipate the next fruit.

```
class QueueItem(object):
    def __init__(self, kind, sprite_size ):
        self.kind = kind
        self._sprite = PreviewSprite( nom=fruit.
        self.y_pos = 0

    def update(self, slot, y):
        x = PREVIEW_SLOT_SIZE * (slot + 0.5)
        self._sprite.position = (x,y,0)
        self._sprite.update(x, y)


class FruitQueue( object ):
    def __init__( self, cnt):
        self._cnt = cnt
        self.y_pos = 0
        self.reset()

    def reset(self):
        self._queue = []
        for _ in range(PREVIEW_COUNT):
            self._add_item()
        self._shift_end_time = None
```

SPRITES.PY

MANAGES GAME GRAPHICS, INCLUDING LOADING AND RENDERING IMAGES OF FRUITS,BACKGROUNDS,AND OTHER VISUAL ELEMENTS. IT COULD DEFINE HOW SPRITES (IMAGES) ARE DRAWN AND ANIMATED.

- DEFINES SPRITE LAYERS
- CREATES ANIMATED EFFECTS
- HANDLES FRUIT OBJECTS.
- IMPLEMENTS EXPLOSIONS.
- OPTIMIZES RENDERING

ABOVE ARE THE FUNCTIONS IMPLEMENTED BY THIS CODE

```python
class SuikaSprite ( pg.sprite.Sprite ):

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self._blink_start = None
        self._fadein_start = None
        self._fadeout_start = None
        self._visibility = VISI_NORMAL

    @property
    def fadein(self):
        return bool(self._fadein_start)

    @fadein.setter
    def fadein(self, activate ):
        if( activate and not self._fadein_start ):
            self._fadein_start = utils.now()
            self._fadeout_start = None
        elif( not activate ):
            self._fadein_start = None


    @property
    def fadeout(self):
        return bool(self._fadeout_start)

    @fadeout.setter
    def fadeout(self, activate):
```

SUIKA_AGENT.PY

THIS IS THE AI AGENT FILE. IT LIKELY CONTAINS THE REINFORCEMENT LEARNING MODEL,DECISION-MAKING LOGIC, OR HEURISTIC RULES THAT ALLOW THE AI TO LEARN HOW TO PLAY SUIKA GAME. THIS FILE IMPLEMENTS RL METHODS USED.

```
class SuikaAgent:
    def __init__(self, state_size=10, action_size=10,
        self.state_size = state_size  # Number of grid
        self.action_size = action_size  # Number of po
        self.lr = learning_rate
        self.gamma = discount_factor
        self.epsilon = epsilon  # Exploration rate
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.997  # Slower decay for
        self.
        self.model_file = "suika_agent.pkl"

        # Training statistics
        self.training_scores = []
        self.training_rewards = []
        self.best_score = 0
        self.total_episodes = 0
        self.episode_rewards = []
        self.episode_scores = []
        self.avg_score = 0
        self.avg_reward = 0
```

(variable) model_file: Literal['suika_ag

- State & Action Space:
- state_size = 10 → The game state is discretized into 10 grid cells.
- action_size = 10 → There are 10 possible drop positions.
- Q-learning Parameters:
- learning_rate = 0.2 → Determines how much new information overrides old Q-values.
- discount_factor = 0.99 → Rewards from future states are considered highly important.
- epsilon = 1.0 → Starts with full exploration.
- epsilon_decay = 0.997 → Gradually reduces exploration over episodes.
- Q-table Initialization:
- self.q_table = defaultdict(lambda: np.zeros(action_size)) → A dictionary storing Q-values for each state-action pair.
- Training Statistics:
- Tracks best score, total episodes, training scores, and rewards.
- Model Persistence:
- self.load_model() → Loads a previously saved model, if available.

SUIKA.PY

THE MAIN GAME LOGIC FILE. IT MIGHT INITIALIZE THE GAME, CREATE THE GAME LOOP, UPDATE FRUIT POSITIONS, HANDLE INPUT, CHECK FOR GAME-OVER CONDITIONS, AND CALL FUNCTIONS FROM OTHER FILES.

User Inputs

- **Left Click** → Drops fruit manually.
- **Right Click** → Shoots fruit.
- **I Key** → Toggles AI.
- **T Key** → Toggles training mode.
- **ESC** → Exits the game.

```
class SuikaWindow(pg.window.Window):
    def __init__(self, width=WINDOW_WIDTH, height=WINDOW_HEIGHT):
        # Initialize all attributes before creating window
        self._is_gameover = False
        self._is_paused = False
        self._autoplay_txt = ""
        self._is_mouse_shake = False
        self._is_benchmark_mode = False
        self._dragged_fruit = None
        self.game_started = False

        # Initialize window
        super().__init__(width=width, height=height, resizable=True

        # Create welcome screen
        self.welcome_screen = WelcomeScreen(width, height, self.sta

        # Initialize game objects
        self._space = pm.Space()
        self._space.gravity = (0, GRAVITY)
        self._bocal = Bocal(space=self._space, **utils.bocal_coords
        self._preview = FruitQueue(cnt=PREVIEW_COUNT)
        self._fruits = ActiveFruits(space=self._space, width=width,
        self._countdown = utils.CountDown()
        self._gui = gui.GUI(window_width=width, window_height=heigh
        self._collision_helper = CollisionHelper(self._space)
        self._autoplayer = Autoplayer()
```

TEST PYGLET.PY

A TEST SCRIPT FOR CHECKING IF PYGLET, A PYTHON LIBRARY FOR RENDERING GRAPHICS, IS WORKING CORRECTLY.
IT MAY LOAD A TEST WINDOW OR DRAW SAMPLE IMAGES TO VERIFY GAME RENDERING.

WHAT THE SCRIPT DOES ??

- OPENS A PYGLET WINDOW.
- CONTINUOUSLY ADDS BALLS THAT FALL DUE TO GRAVITY.
- DISPLAYS AN L-SHAPED OBJECT THAT ROTATES SLIGHTLY.
- REMOVES BALLS THAT FALL OFF THE SCREEN.

THIS IS A SIMPLE PHYSICS SIMULATION USING PYGLET + PYMUNK

```
    )

class Ball( object ):
    def __init__(self, body, pg_shape ):
        self.body = body

        # Cree la forme pyglet associée à l'objet pymunk
        self.pg_shape = pg_shape

    def is_offscreen(self) -> bool :
        return self.body.position.y < 0

    def position_changed(self):
        self.pg_shape.x , self.pg_shape.y = self.body.position
        #self.pg_shape.y = self.body.position.y

    def deallocate(self):
        circle = self.body.shapes
        assert len(circle)==1
        self.body.space.remove( self.body, *circle )


class Lshape( object ):
```

UTILS.PY

CONTAINS UTILITY/HELPER FUNCTIONS USED THROUGHOUT THE PROJECT, SUCH AS MATHEMATICAL CALCULATIONS, FILE HANDLING, OR OTHER REUSABLE FUNCTIONS THAT DON'T FIT INTO A SINGLE MODULE.

Summary
- Speedmeter: Measures the speed of game actions.
- CountDown: Handles countdown timers for game-over conditions.
- bocal_coords(): Calculates the play area dimensions.
- Commented-out debugging tools for tracking in-game object counts.

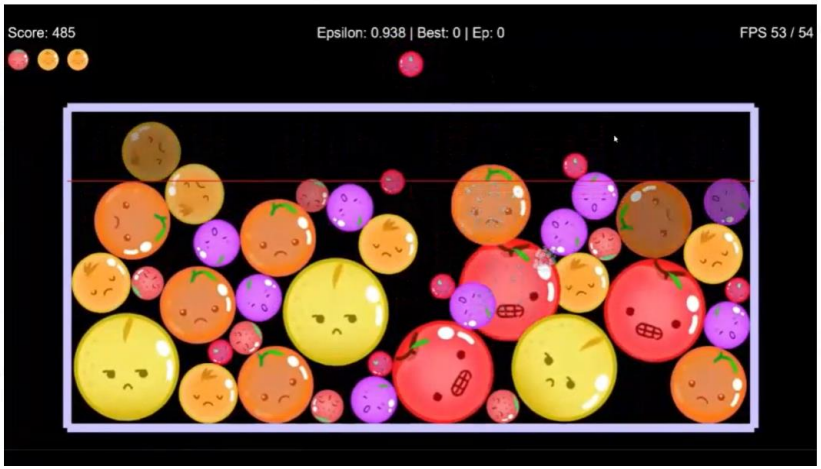
This file mostly handles game timing, performance measurement, and UI layout calculations.

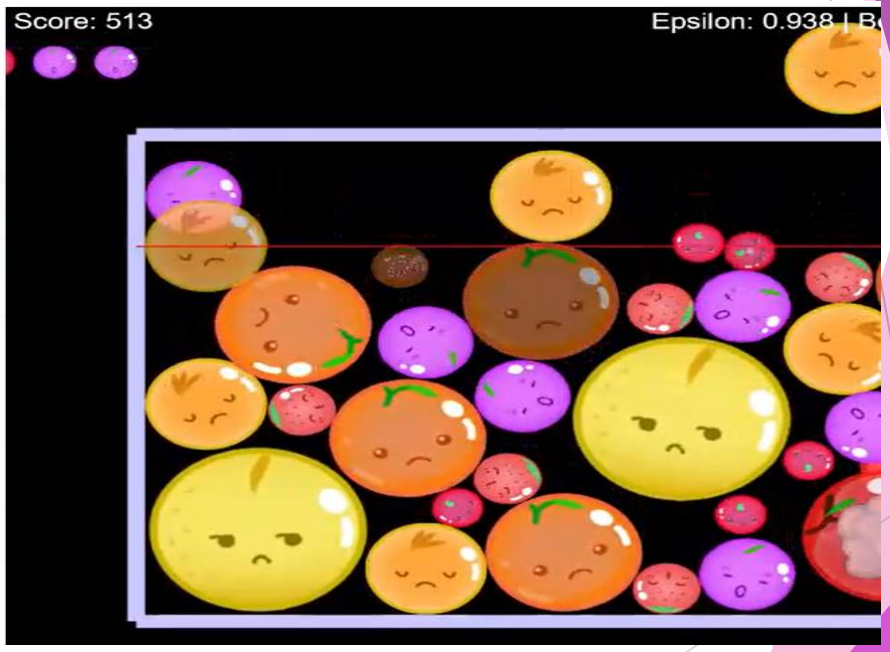
```
class Speedmeter(object):
    def __init__(self, bufsize=DEFAULT_BUFSIZE ):
        self._deltas = collections.deque( maxlen=bufsize )
        self._value = 0.0
        self._last_tick = None
        self._last_refresh = 0

    def tick_rel(self, dt):
        self._deltas.append(dt)

    def tick(self):
        current = time.perf_counter()
        if( self._last_tick ):
            last = self._last_tick
            self._deltas.append( current-last )
        self._last_tick = current

    @property
    def value(self):
        if( len(self._deltas) == 0 ):
            return 0
        if( now() - self._last_refresh >= SPEEDMETER_UPDAT
            s = sum(self._deltas)
            if( s>0 ):
                self._value = len(self._deltas) / s
                self._last_refresh = now()
        return self._value
```





DIFFERENT STAGES OF AI AGENT PLAYING GAME AND LEARNING


THANK YOU!